

DeepWonder3D: Rapid robust high-fidelity 3D neuronal extraction from multi-view projections

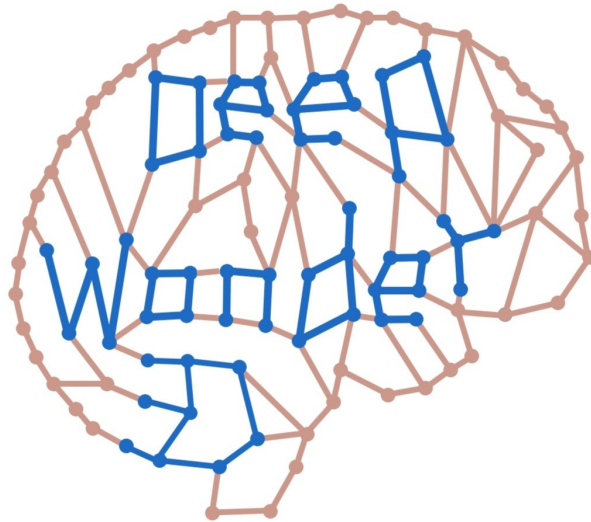


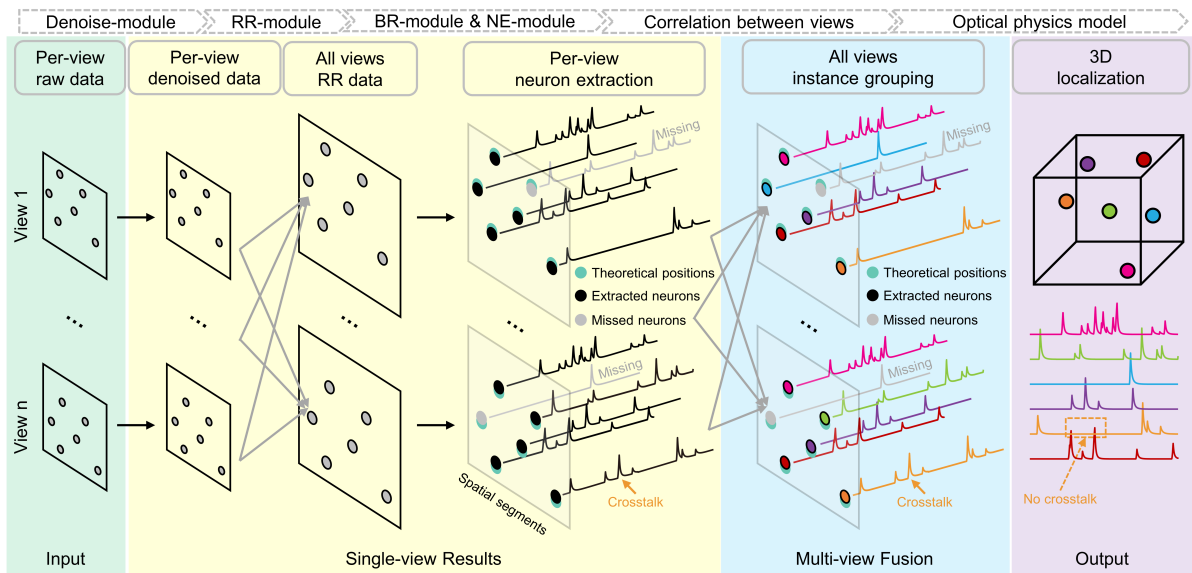
Table of Contents

- [About DeepWonder3D](#)
- [Quick Start](#)
- [Our Environment](#)
- [Installation](#)
- [Usage](#)

About DeepWonder3D

DeepWonder3D is a general pipeline designed for rapid and robust neuronal extraction from 3D calcium imaging datasets. Unlike conventional methods that operate voxel by voxel, DeepWonder3D processes multi-view projections—either digitally or optically generated via specific point spread functions (PSFs)—making it broadly compatible with diverse imaging techniques, including point-scanning microscopy, light-field microscopy, and synthetic aperture two-photon microscopy.

The pipeline integrates **denoising**, **resolution registration**, **background removal**, **neuronal extraction**, and **multi-view fusion** into a unified workflow optimized for large-scale high-resolution datasets contaminated by noise and scattering. DeepWonder3D enables accurate localization of neurons in 3D space and extracts high-fidelity temporal calcium traces, all with notably improved processing efficiency. Its effectiveness has been validated on both simulated data and real recordings from a hybrid two-photon/light-field imaging system.



Quick Start

We provide two versions of **DeepWonder3D**:

1. **Python source code version** (primarily implemented in [PyTorch](#)), available directly in this repository [Deepwonder3D_pytorch](#). Installation instructions are provided in the following sections. Typically, setting up the required environment takes under 10 minutes on a standard machine, assuming all files are pre-downloaded. However, total installation time may vary depending on your network speed, as certain commands (e.g., `conda install`) require downloading packages.
2. **GUI version**, which can be downloaded [here](#). This version can be used immediately without environment setup.

Once the installation is complete, comprehensive usage documentation can be found [here](#). To facilitate a quick hands-on experience with **DeepWonder3D**, we include a `datasets` folder in `Deepwonder3D_pytorch` as a demo for users to try out.

Our Environment

- Ubuntu 20.04
- Python 3.8
- Pytorch 2.0.1
- TensorRT 8.6.1
- CUDA 11.8
- cuDNN 8.9.0

Installation

1. Install [Anaconda](#), [CUDA 11.8](#), and [cuDNN 8.9.0](#)
2. Create a virtual environment and install PyTorch 2.0.1.

```
conda create -n dw3d python=3.8
conda activate dw3d
conda install pytorch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 pytorch-
cuda=11.8 -c pytorch -c nvidia
```

3. (recommended, but optional) Login / Register a nvidia account and download the TAR package for TensorRT [here](#).

4. (recommended, but optional) Install the TensorRT according to the [official installation guide](#).
5. Download the source code and use DeepWonder3D

[Important Notes !!!]

1. The following commands can be used to verify that your installation works well.

```
python3
>>> import tensorrt
>>> print(tensorrt.__version__)
>>> assert tensorrt.Builder(tensorrt.Logger())
```

2. If the above test fails, one of the possible reasons is that the global **PATH is not set** correctly. Suppose your TensorRT is in the folder `/data/user/lib64/TensorRT-8.6.1.6`, then add the following to `.bashrc` or `.profile`

```
# >>> tensorRT initialize >>>
export CUDA_MODULE_LOADING=LAZY
export LD_LIBRARY_PATH="/data/user/lib64/TensorRT-
8.6.1.6/lib:$LD_LIBRARY_PATH"
export PATH="/data/user/lib64/TensorRT-8.6.1.6/bin:$PATH"
# <<< tensorRT initialize <<<
```

If the TensorRT folder moved to another place, don't forget to change the path correspondingly.

3. Installing pytorch requires aligning CUDA version + Python version, while installing TensorRT requires aligning CUDA version + cuDNN version + Python version. For example, the following screenshot of [Installation Guide for TensorRT 8.6.1](#) explicitly tells the CUDA versions and cuDNN versions that TensorRT 8.6.1 is compatible with.

2. Getting Started

Ensure you are familiar with the following installation requirements and notes.

- If you are using the TensorRT Python API and CUDA-Python isn't already installed on your system, refer to the [NVIDIA CUDA-Python Installation Guide](#).
- Ensure you are familiar with the [NVIDIA TensorRT Release Notes](#).
- Verify that you have the NVIDIA CUDA™ Toolkit installed. If CUDA is not already installed, review the [NVIDIA CUDA Installation Guide](#) for instructions on how to install the CUDA Toolkit. The following versions are supported:
 - [12.1 update 1](#)
 - [12.0 update 1](#)
 - [11.8](#)
 - [11.7 update 1](#)
 - [11.6 update 2](#)
 - [11.5 update 2](#)
 - [11.4 update 4](#)
 - [11.3 update 1](#)
 - [11.2 update 2](#)
 - [11.1 update 1](#)
 - [11.0 update 1](#)
- cuDNN is now an optional dependency for TensorRT and is only used to speed-up a small number of layers. If you require cuDNN, then verify that you have cuDNN installed. Review the [NVIDIA cuDNN Installation Guide](#) for more information. TensorRT 8.6.1 supports [NVIDIA cuDNN 8.9.0](#). cuDNN is not used by the lean or dispatch runtimes.
- Some Python samples require [TensorFlow 2.5.1](#), such as [efficientdet](#) and [efficientnet](#). In addition, the deprecated UFF model export from TensorFlow requires [TensorFlow 1.15.5](#).
- The PyTorch examples have been tested with [PyTorch 1.13.1](#), but may work with older versions.
- The ONNX-TensorRT parser has been tested with [ONNX 1.12.0](#) and supports opset 16.

The combination of versions we offered here will work well. But if you want to use different versions of Python, CUDA, cuDNN, Pytorch, and TensorRT, **make sure that their versions are compatible with each other.**

4. In fact, our method is also compatible with Windows system. We have tested it on Win10 and it works well. The main difference during environment installation is
 - Download the ZIP package of TensorRT instead of TAR package
 - Setting the environment variables in a different way

To add/change an *environment variable* permanently in Windows (so that it is available to ALL the Windows' processes/users and stayed across boots):

1. Launch "Control Panel"

2. "System"
3. "Advanced system settings"
4. Switch to "Advanced" tab
5. "Environment variables"
6. Choose "System Variables" (for all users)
7. To *add* a new environment variable:
 1. Choose "New"
 2. Enter the variable "Name" and "Value". Instead of typing the "value" and making typo error, I suggest that you use "Browse Directory..." or "Browse File..." button to retrieve the desired directory or file.
8. To *change* an existing environment variable:
 1. Choose "Edit"
 2. Enter the new "Value". Instead of typing the "value" and making typo error, I suggest that you use "Browse Directory..." or "Browse File..." button to retrieve the desired directory or file.

You need to **RE-START** CMD for the new setting to take effect!

(cited from [here](#))

Usage

Following a successful installation, comprehensive usage documentation can be found [here](#).

While the full documentation is extensive, users only need three commands to get up and running.

```
conda activate dw3d
cd Deepwonder3D_pytorch
python main_pipeline_3d.py
```